

Bàn về ứng dụng tìm kiếm bộ phận kết hợp thuật toán hiệu quả trong các bài toán tìm kiếm

Lou Tiancheng
Trường Trung học số 14 Hàng Châu, Chiết Giang

2004

Nguồn gốc: A brief discussion of partial search + efficient algorithms in search problems

IOI China candidate team papers / 2004 / Lou Tiancheng

Abstract

Bài viết bắt đầu từ việc tìm kiếm trong các bài toán ghép cặp có ràng buộc vị trí. Qua phân tích bài *Milk Bottle Data*, bài viết đưa ra một kiểu tìm kiếm không thông thường của tìm kiếm theo chiều sâu: **tìm kiếm bộ phận + thuật toán hiệu quả**. Sau đó, thông qua ứng dụng của tìm kiếm bộ phận trong hai bài *Triangle Construction* và *Phá trận liên hoàn* (智破连环阵), bài viết thảo luận các cách tối ưu chủ yếu dùng chung cho phương pháp này, đồng thời phân tích bản chất của nó để chỉ ra vì sao nó hiệu quả, cũng như các điều kiện và hạn chế cần thỏa khi áp dụng.

Từ khóa: tìm kiếm bộ phận; thuật toán hiệu quả.

1 Mở đầu

Trong nhiều bài toán, nếu có thể xây dựng mô hình toán học thì ta nên cố gắng dùng phương pháp giải tích để xử lý, vì mô hình đơn giản phản ánh rõ hơn quan hệ giữa các đối tượng.

Tuy nhiên, không phải bài toán nào cũng có thể xây dựng một mô hình toán học đơn giản. Khi đó ta buộc phải dùng phương pháp tìm kiếm, tức liệt kê mọi khả năng để tìm nghiệm khả thi hoặc nghiệm tối ưu.

Vì tìm kiếm dựa trên liệt kê, nên nó thường gắn với hiệu suất thấp. Có lúc khối lượng tính toán của tìm kiếm quá lớn, khiến việc xử lý trở nên rất nặng.

Do đó ta cần dùng nhiều kỹ thuật để nâng cao hiệu suất. Cắt tỉa theo tính khả thi, cắt tỉa theo tính tối ưu và điều chỉnh thứ tự tìm kiếm đều rất hữu ích; nhờ chúng, hiệu suất tìm kiếm có thể tăng lên đáng kể.

Nhưng với một số bài, các tối ưu thông thường ấy khó có đất dụng võ. Khi đó ta phải dùng các phương pháp tìm kiếm không thông thường. Trong bài này ta thảo luận một trong số đó: **tìm kiếm bộ phận + thuật toán hiệu quả**.

2 Bài dẫn

Có N vật phẩm và N vị trí. Với mỗi vật phẩm, ta biết tập các vị trí mà nó có thể đặt vào; cần tìm một song ánh giữa vật phẩm và vị trí. Ngoài ra còn có các ràng buộc giữa vật phẩm và vị trí, chẳng hạn: nếu

vật phẩm 1 đặt ở vị trí 3 thì vật phẩm 2 không được đặt ở vị trí 1. Yêu cầu có thể là tìm một nghiệm khả thi, hoặc gán trọng số cho mỗi cách tương ứng rồi tìm nghiệm tối ưu thỏa điều kiện.

Do quan hệ ràng buộc giữa các đối tượng rất phức tạp, rất khó xây dựng một quan hệ đồ thị hai phía đơn giản, hoặc dùng luồng mạng để giải.

Trước một loạt bài toán kiểu này, thông thường ta chỉ còn cách tìm kiếm. Vấn đề là tìm kiếm thế nào và tối ưu ra sao.

2.1 Phân tích đơn giản

Nếu liệt kê vị trí của từng vật phẩm rồi kiểm tra, độ phức tạp thời gian là $\mathcal{O}(N!)$. Thoạt nhìn dường như cũng chỉ có thể làm như vậy.

2.2 Phân tích thêm

Xét một ví dụ với $N = 6$. Có bốn ràng buộc khác, mỗi ràng buộc dạng (a, b, c, d) nghĩa là nếu a đặt ở b thì c không được đặt ở d :

1 3 5 6
2 2 5 3
3 1 4 1
3 2 6 2

Sau đó ta nhận ra: một khi đã xác định vị trí của vật phẩm 3 và 5, giữa vị trí của bốn vật phẩm còn lại không còn quan hệ ràng buộc nào nữa. Khi đó vị trí của bốn vật phẩm còn lại có thể giải bằng ghép cặp.

Từ đây ta thấy một mô hình tìm kiếm mới: **tìm kiếm bộ phận**.

Tìm kiếm bộ phận: tìm kiếm trên một phần biến để quan hệ giữa các biến còn lại được đơn giản hóa; sau đó dùng một số thuật toán hiệu quả, thường là ghép cặp, giải hệ phương trình, tham lam, quy hoạch động, v.v., để hoàn thành phần còn lại của bài toán.

Với bài dẫn trên, ta tìm kiếm trước vị trí của một số lượng vật phẩm nhất định (không phải toàn bộ), làm cho quan hệ giữa các vật phẩm trong phần còn lại trở thành quan hệ đồ thị hai phía, rồi dùng ghép cặp hai phía để xử lý các vật phẩm còn lại.

2.3 Bản chất

Trong ví dụ trên, nếu sau khi biết vị trí của 3 và 5 mà ta không dùng ghép cặp, thì thực chất ta đang dùng tìm kiếm để tìm một ghép cặp; hiệu suất tất nhiên sẽ không cao.

Tìm kiếm bộ phận tạo điều kiện cho các thuật toán hiệu quả khác, chẳng hạn tạo ra quan hệ hai phía trong ví dụ trên; còn thuật toán hiệu quả thay thế tìm kiếm để hoàn thành nhanh phần việc còn lại.

Phương pháp tìm kiếm bộ phận phát huy đầy đủ ưu thế của cả tìm kiếm lẫn các thuật toán hiệu quả khác. Ưu thế của tìm kiếm là phạm vi áp dụng rộng, có thể vượt qua tình huống phức tạp; ưu thế của các thuật toán khác là hiệu suất cao. Hai phía thúc đẩy nhau, đồng thời bù đắp nhược điểm của nhau. Đây chính là chìa khóa thành công của phương pháp.

Tìm kiếm bộ phận kết hợp các thuật toán hiệu quả khác đã được dùng trong rất nhiều bài. Dưới đây ta dùng vài ví dụ để thảo luận cách áp dụng và các kỹ thuật tối ưu của kiểu tìm kiếm này.

3 Ví dụ: Milk Bottle Data

3.1 Mô tả bài toán

Milk Bottle Data xuất hiện tại ACM/ICPC Asia Regional Shanghai 1996.

Một hộp được chia thành lưới $N \times N$; mỗi ô có thể chứa một chai sữa hoặc không chứa gì. Ông Smith ghi lại tình trạng sữa của từng hàng từ trái sang phải, và cũng ghi lại tình trạng sữa của từng cột từ trên xuống dưới. Mỗi bản ghi gồm N chữ số: 0 nghĩa là không có sữa, 1 nghĩa là có sữa. Không may, thứ tự của $2N$ bản ghi đã bị xáo trộn, và một vài chữ số cũng bị mờ.

Bây giờ ông Smith nhờ bạn khôi phục tình trạng sữa ban đầu trong hộp.

Dữ liệu vào. Dòng đầu là số nguyên N . Tiếp theo là $2N$ dòng, mỗi dòng có N chữ số: 0 nghĩa là chắc chắn không có sữa, 1 nghĩa là chắc chắn có sữa, 2 nghĩa là không xác định.

Ví dụ.

```
input
5
01210
21120
21001
12110
12101
12101
00011
22222
11001
10010
output
9 8 6 2 7
4 1 0 1 1 0
10 1 0 0 1 0
1 0 1 1 1 0
3 0 1 0 0 1
5 1 1 1 0 1
```

Ràng buộc dữ liệu: $1 \leq N \leq 10$.

3.2 Phân tích ban đầu

Quan hệ ràng buộc giữa hàng và cột rất phức tạp, khó tìm được thuật toán đa thức. Luồng mạng cũng không dễ áp dụng.

Ta có thể dùng tìm kiếm theo chiều sâu với $(2N)!$ phương án: lần lượt liệt kê số hiệu bản ghi cho từng hàng và từng cột, rồi kiểm tra có sinh mâu thuẫn hay không.

Gọi chữ số thứ j của bản ghi thứ i là $A[i, j]$. Nếu hàng a chọn bản ghi x , cột b chọn bản ghi y , thì điều kiện mâu thuẫn là

$$(A[x, b] \neq 2) \wedge (A[y, a] \neq 2) \wedge (A[x, b] \neq A[y, a]).$$

3.3 Phân tích hiệu suất thứ nhất

Vì $N \leq 10$, giá trị $(20)! \cdot 10$ lớn tới

$$24329020081766400000.$$

Thử vài tối ưu cơ bản:

- Cắt tỉa khả thi: ngoài kiểm tra sớm, gần như không còn tối ưu nào khác.
- Cắt tỉa tối ưu: bài toán không phải bài tối ưu.
- Điều chỉnh thứ tự tìm kiếm: nếu mỗi lần tìm kiếm nhánh có ít khả năng nhất thì hiệu quả vẫn khá tốt. Nhưng trường hợp xấu nhất vẫn phải tính $(20)!$, khó gọi là một thuật toán khả thi tốt.

3.4 Tìm kiếm bộ phận + ghép cặp

Ta nhận thấy giữa các hàng với nhau và giữa các cột với nhau không có ràng buộc; mọi ràng buộc đều nằm giữa hàng và cột. Vậy ta có thể chỉ tìm kiếm các hàng không? Câu trả lời là có.

Nếu đã biết giá trị của từng hàng, thì giữa các cột không còn quan hệ ràng buộc. Khi đó ta có thể dùng ghép cặp để tìm một nghiệm khả thi.

Cách dựng đồ thị như sau. Phía trái của đồ thị hai phía có N đỉnh, biểu diễn N bản ghi độ dài N chưa được chọn; phía phải có N đỉnh, biểu diễn N cột. Nếu đưa bản ghi ứng với đỉnh trái thứ i vào cột j không gây mâu thuẫn ở bất kỳ hàng nào của cột j , thì thêm một cạnh giữa đỉnh trái thứ i và đỉnh phải thứ j .

Ghép cặp hai phía giải được trong $\mathcal{O}(N^3)$, hiệu quả hơn nhiều so với tìm kiếm mù.

3.5 Phân tích hiệu suất thứ hai

Khi $N = 10$, khối lượng tính toán trở thành

$$P(20, 10) \cdot (20 \cdot 10 \cdot 10) = 13408850145600000.$$

So với tìm kiếm đơn giản, hiệu suất đã được cải thiện rất nhiều. Vì chỉ cần tìm một nghiệm là chương trình có thể kết thúc, cách này có thể nhanh chóng vượt qua toàn bộ dữ liệu kiểm thử.

Ta có thể so sánh hai thuật toán như sau. Nếu tìm kiếm thông thường là tìm kiếm hàng trước rồi tìm kiếm cột, thì sau khi tìm kiếm xong hàng, nó đang dùng tìm kiếm để tìm một ghép cặp. Độ phức tạp của tìm kiếm là $\mathcal{O}(N!)$, còn độ phức tạp của ghép cặp là $\mathcal{O}(N^3)$. Hiệu suất khác nhau là hiển nhiên.

Thuật toán tìm kiếm này vẫn có thể dùng các tối ưu thông thường. Phương pháp tìm kiếm đặc biệt và các tối ưu thông thường không mâu thuẫn với nhau. Thuật toán này còn có nhiều tối ưu riêng; ta tiếp tục qua hai ví dụ.

4 Ví dụ: Triangle Construction

4.1 Mô tả bài toán

Triangle Construction là bài trong ZJU Monthly Contest.

Một tam giác đều cạnh N có thể chia thành N^2 tam giác đều nhỏ cạnh 1. Trong hình gốc, các tam giác nhỏ được ghép lại sao cho hai cạnh kề nhau phải có cùng màu. Bài toán cho màu của ba cạnh của từng tam giác trong N^2 tam giác nhỏ; hãy xác định có thể dùng chúng để tạo thành một tam giác đều cạnh N hay không.

Ở đây bài viết chỉ xét trường hợp $N = 4$.

4.2 Phân tích

Tìm kiếm trực tiếp có khối lượng $16! = 20922789888000$, quá lớn.

Thực ra đây cũng là một bài ghép cặp có ràng buộc vị trí với 16 vật thể. Khác với bài dẫn ở chỗ: ràng buộc giữa các vị trí là xác định, vì vậy ta phải tận dụng đầy đủ đặc điểm này.

Nếu dùng thuật toán tìm kiếm bộ phận + ghép cặp, ta nên tìm kiếm thế nào? Đương nhiên ta cần chọn cách có hiệu suất cao nhất.

Giả sử số tam giác được liệt kê là T , khối lượng tính toán là

$$P(16, T) \cdot 256 \cdot (16 - T).$$

Các giá trị trong bài gốc được liệt kê như sau:

T	Khối lượng	T	Khối lượng
0	4096	9	7439214182400
1	61440	10	44635285094400
2	860160	11	223176425472000
3	11182080	12	892705701888000
4	134184960	13	2678117105664000
5	1476034560	14	5356234211328000
6	14760345600	15	5356234211328000
7	132843110400	16	?
8	1062744883200		

Kết luận: khối lượng tính toán tăng dần theo T . Với dạng bài này, trong trường hợp thông thường, số biến liệt kê càng ít càng tốt.

Tối ưu. Chọn các biến liệt kê tốt, sao cho số biến cần liệt kê ít nhất có thể.

Với bài này, ta có thể liệt kê sáu tam giác ở phần được tô bóng trong hình gốc; mười tam giác còn lại đều có thể dựng thành quan hệ hai phía đơn giản. Khối lượng tính toán là

$$P(16, 6) \cdot 256 \cdot 10 = 14760345600.$$

Cộng thêm các tối ưu thông thường, hiệu suất được nâng lên rất nhiều.

4.3 Nhận xét

Khi chọn biến tìm kiếm, nguyên tắc chung là làm cho số biến tìm kiếm ít nhất có thể, vì thuật toán hiệu quả có độ phức tạp đa thức, còn tìm kiếm là dạng NP.

Cách chọn đôi khi có thể dựa trên phân tích, như hai bài vừa nêu; nhưng trong một số bài, chương trình cần tự xác định biến nào phải tìm kiếm.

5 Ví dụ: Phá trận liên hoàn (NOI 2003)

Đây là một ví dụ kinh điển về tối ưu phương pháp tìm kiếm bộ phận.

5.1 Mô tả bài toán

Sau khi tiêu tốn hàng chục tỷ, nước B cuối cùng đã chế tạo ra vũ khí mới: Zenith Protected Linked Hybrid Zone, gọi là *trận liên hoàn*, và tuyên bố đây là một vũ khí tự phát thông minh không thể đánh bại. Nhưng sau khi trinh sát, nước A phát hiện trận liên hoàn thực ra gồm M vũ khí độc lập. Các vũ khí này được đánh số $1, 2, \dots, M$. Mỗi vũ khí có hai trạng thái: trạng thái tự vệ vô địch và trạng thái tấn công. Ban đầu, vũ khí số 1 ở trạng thái tấn công, các vũ khí khác đều ở trạng thái tự vệ vô địch. Về sau, mỗi khi vũ khí số i ($1 \leq i < M$) bị tiêu diệt, sau 1 giây vũ khí số $i + 1$ tự động chuyển từ trạng thái tự vệ vô địch sang trạng thái tấn công. Khi vũ khí số M bị tiêu diệt, trận liên hoàn đất đổ này bị phá hủy hoàn toàn.

Để đánh bại nước B, Bộ trưởng quân sự nước A định dùng loại vũ khí rẻ nhất: bom. Sau thời gian dài thăm dò chính xác, các nhà quân sự nước A nắm được tọa độ phẳng của M vũ khí trong trận liên hoàn; dựa vào đó họ chọn n điểm và đặt bom hẹn giờ đặc biệt tại các điểm này. Các quả bom được đánh số $1, 2, \dots, n$. Bán kính tác dụng của mỗi quả bom đều là k , và nó nổ liên tục trong 5 phút. Trong 5 phút đó, mỗi quả bom có thể tức thời tiêu diệt bất kỳ số vũ khí B nào đang ở trạng thái tấn công và có khoảng cách đường thẳng đến nó không vượt quá k . Tương tự trận liên hoàn, ban đầu bom số a_1 nổ liên tục trong 5 phút, sau đó bom số a_2 nổ liên tục trong 5 phút, tiếp theo là a_3 , v.v., cho đến khi trận liên hoàn bị phá hủy. Khi một quả bom nổ, các quả bom chưa nổ khác đều còn được che giấu dưới đất và không bị bom của phe mình phá hủy.

Rõ ràng việc chọn $a_1, a_2, a_3, \dots$ rất quan trọng. Một dãy tốt có thể phá hủy trận liên hoàn chỉ với ít bom; một dãy xấu có thể dùng hết mọi quả bom mà vẫn chưa phá hủy được. Hãy quyết định một dãy $a_1, a_2, a_3, \dots$ sao cho trận liên hoàn bị phá hủy trong thời gian bom số a_x nổ. Giá trị x cần càng nhỏ càng tốt.

Dữ liệu vào. Tập `zplhz.in` có dòng đầu gồm ba số nguyên M, n, k ($1 \leq M, n \leq 100, 1 \leq k \leq 1000$), lần lượt là số vũ khí của nước B, số bom của nước A, và phạm vi tấn công của bom. Tiếp theo là M dòng, mỗi dòng gồm một cặp số nguyên x_i, y_i ($0 \leq x_i, y_i \leq 10000$), là tọa độ của vũ khí số i . Sau đó là n dòng, mỗi dòng gồm một cặp số nguyên u_i, v_i ($0 \leq u_i, v_i \leq 10000$), là tọa độ của bom số i . Dữ liệu vào bảo đảm hợp lệ và có nghiệm.

Trong dữ liệu kiểm thử, các giá trị x_i, y_i, u_i, v_i được sinh ngẫu nhiên.

Dữ liệu ra. Tập `zplhz.out` có dòng đầu là một số nguyên x , tức số bom thực sự được dùng. Dòng thứ hai gồm x số nguyên, lần lượt là $a_1, a_2, \dots, a_x$.

5.2 Phân tích ban đầu

Bom I của nước A bắn trúng vũ khí J của nước B khi và chỉ khi

$$(u[I] - x[J])^2 + (v[I] - y[J])^2 \leq r^2.$$

Khó tìm được một thuật toán đa thức để tìm nghiệm tối ưu. Với dạng bài này, thông thường ta chỉ có chiến lược tìm kiếm.

Phân tích thêm, mỗi quả bom nhất định tiêu diệt một đoạn liên tiếp theo chỉ số trong các vũ khí của nước B. Thời gian 5 phút chỉ nói rằng một quả bom có thể tiêu diệt tùy ý nhiều vũ khí B có chỉ số liên tiếp.

5.3 Tìm kiếm bộ phận

Để dùng tìm kiếm bộ phận trong bài này, ta cần một phép chuyển hóa: giả sử đã chia các vũ khí B theo chỉ số thành x đoạn $[S_i, T_i]$, trong đó $S_1 = 1, T_i \geq S_i$ và $S_{i+1} = T_i + 1$. Sau đó kiểm tra liệu có thể chọn

x quả bom trong N quả bom của nước A, mỗi quả tiêu diệt một đoạn tương ứng hay không.

Thực chất ta chia tìm kiếm thành hai phần: trước tiên tìm kiếm cách chia vũ khí B theo chỉ số thành x đoạn; sau đó tìm kiếm xem có thể chọn x quả bom từ N quả bom của nước A, mỗi quả tiêu diệt một đoạn, hay không. Phần thứ hai có thể giải bằng ghép cặp.

Đặt $C[S][T][I]$ biểu diễn bom I của nước A có thể tiêu diệt toàn bộ vũ khí B từ $S, S + 1, \dots, T - 1, T$ hay không. Khi đó:

$$\begin{aligned} C[S][S][I] &= ((u[I] - x[S])^2 + (v[I] - y[S])^2 \leq R^2), \\ C[S][T][I] &= C[S][T - 1][I] \wedge C[T][T][I] \quad (S < T). \end{aligned}$$

Tính C mất $\mathcal{O}(N^3)$.

Dựng đồ thị: phía trái có x đỉnh, biểu diễn x đoạn của vũ khí B sau khi chia; phía phải có N đỉnh, biểu diễn N quả bom của nước A. Có cạnh từ đỉnh trái thứ i đến đỉnh phải thứ j khi $C[S_i][T_i][j]$ đúng.

Nhiệm vụ của tìm kiếm là chia vũ khí B theo chỉ số thành một số đoạn, rồi dùng ghép cặp hai phía để kiểm tra.

5.4 Phân tích hiệu suất thứ nhất

Khung tìm kiếm cơ bản đã có. Tuy dữ liệu được sinh ngẫu nhiên, số cách chia M vũ khí B vẫn rất nhiều, đôi khi có thể cao tới 2^m . Thời gian khó chịu đựng; nếu dùng giới hạn thời gian cứng thì tính đúng đắn bị ảnh hưởng, hiệu quả sẽ không tốt.

Chỉ có 4 bộ dữ liệu có thể ra nghiệm trong giới hạn thời gian; với 6 bộ còn lại, nếu dùng giới hạn thời gian thì có thêm 1 bộ có thể nhận được nghiệm tối ưu.

5.5 Các tối ưu đầu tiên

Có thể phân tích tối ưu từ hai mặt: tính khả thi và tính tối ưu.

Tối ưu 1: cắt tỉa tối ưu. Nếu bom của nước A được phép dùng lặp lại, đặt

$$Dist[i] = \text{số bom ít nhất cần dùng để tiêu diệt vũ khí B từ } i \text{ đến } m.$$

Giá trị này có thể tính bằng quy hoạch động:

$$Dist[m + 1] = 0,$$

$$Dist[i] = \min\{Dist[j] + 1 \mid Can[i][j - 1][k], 1 \leq k \leq n, i < j \leq n + 1\}.$$

Tính $Dist$ mất $\mathcal{O}(N^3)$. Từ đó có điều kiện cắt tỉa tối ưu:

$$\text{nếu số bom đã dùng} + Dist[\text{vũ khí B tiếp theo}] \geq \text{nghiệm tốt nhất hiện có, thì cắt tỉa.}$$

Tối ưu 2: cắt tỉa khả thi. Với phương pháp tìm kiếm này, thường có hai tối ưu khả thi rất hiệu quả:

1. Kiểm tra sớm xem ghép cặp có thể thành công hay không, tránh tìm kiếm thừa.
2. Mỗi lần ghép cặp có thể mở rộng từ ghép cặp trước đó, không cần bắt đầu lại từ đầu.

Nếu cách chia hiện tại đã không thể ghép cặp thành công, không cần tiếp tục tìm kiếm. Chỉ cần mỗi khi tìm kiếm thêm một đoạn mới thì lập tức dùng ghép cặp để kiểm tra. Mỗi lần tìm ghép cặp chỉ cần mở rộng từ nền tảng cũ.

Sau hai tối ưu trên, hiệu suất chương trình tăng đáng kể. Trong 10 bộ kiểm thử, có 8 bộ ra nghiệm trong giới hạn thời gian; với 2 bộ còn lại, nếu dùng giới hạn thời gian thì có 1 bộ có thể nhận được nghiệm tối ưu.

5.6 Tối ưu thêm

Tối ưu 2 tuy loại bỏ nhiều cách chia không cần thiết, nhưng lãng phí không ít thời gian khi kiểm tra. Vì vậy, khi liệt kê độ dài đoạn chia tiếp theo, ta có thể dựa trên cách chia và tình trạng ghép cặp trước đó, tức các cạnh đã được ghép, để dùng tìm kiếm theo chiều rộng $\mathcal{O}(n^2)$ tính độ dài lớn nhất $maxL$ của đoạn kế tiếp. Rõ ràng mọi độ dài của đoạn kế tiếp trong $[1, maxL]$ đều chắc chắn tìm được một ghép cặp khả thi. Như vậy vừa tiết kiệm thời gian kiểm tra, vừa có thể liệt kê độ dài đoạn từ dài đến ngắn, giúp chương trình nhanh chóng tiến gần nghiệm tối ưu và đồng thời tăng hiệu quả của điều kiện cắt tỉa 1.

Trước hết cần tính $MaxT$:

$$MaxT[i][S] = \text{chỉ số lớn nhất mà bom } i \text{ có thể tiêu diệt nếu bắt đầu từ } S.$$

Nếu bom i không tiêu diệt được S , thì $MaxT[i][S] = S - 1$.

Ta có thể dùng quy hoạch động để tính $MaxT[i][S]$:

$$MaxT[i][S] = \begin{cases} S - 1, & \text{nếu bom } i \text{ không tiêu diệt được } S, \\ MaxT[i][S + 1], & \text{nếu bom } i \text{ tiêu diệt được } S. \end{cases}$$

và $MaxT[i][m + 1] = m$. Thời gian tính $MaxT$ là $\mathcal{O}(N^2)$.

Cách cài đặt cụ thể xét n đỉnh phía phải của đồ thị hai phía, tức n quả bom. Nếu đỉnh i chưa được ghép, ta xem i là có thể sử dụng. Với một đỉnh i đã được ghép, nếu tồn tại một đường xen kẽ đi từ một đỉnh chưa ghép nào đó đến i , và i là đỉnh ngoài trên đường ấy, thì i cũng được xem là có thể sử dụng.

Ta cần tính các đỉnh đã ghép mà những đường xen kẽ xuất phát từ đỉnh chưa ghép có thể đạt tới. Chỉ cần từ mỗi đỉnh chưa ghép chạy tìm kiếm theo chiều rộng, mất $\mathcal{O}(N^2)$. Cần chú ý tránh lặp: nếu một đỉnh đã ghép đã được xác định là có thể sử dụng, thì không cần mở rộng nó lần nữa, vì khi xác định nó là đỉnh có thể sử dụng ta đã mở rộng từ nó; mở rộng lại chỉ tạo ra lặp thừa.

Do đó

$$MaxL = \max\{MaxT[i][S] \mid i \text{ có thể sử dụng}\}.$$

5.7 Phân tích hiệu suất thứ ba

Sau các tối ưu trên, toàn bộ dữ liệu đều ra nghiệm ngay lập tức, và mọi kết quả đều là nghiệm tối ưu. Ngay cả với dữ liệu ngẫu nhiên $n = 200$, chương trình cũng có thể ra nghiệm tức thời; điều này cho thấy hiệu suất đã được cải thiện rất nhiều.

5.8 Tinh chỉnh thêm

Còn hai tối ưu nữa, đôi khi hiệu quả không tốt nhưng vẫn đáng nhắc tới.

Tối ưu 3: phân nhánh cận. Cách này có thể tăng hiệu quả của điều kiện cắt tỉa 1, nhưng khi nghiệm tối ưu khác xa $Dist[1]$, nó sẽ lãng phí một lượng thời gian nhất định.

Tối ưu 4: cải thiện giá trị $Dist[i]$. Trong tối ưu 1, giá trị $Dist[i]$ đôi khi không tối ưu. Qua thử nghiệm, nếu $Dist[i]$ lệch nghiêm trọng tối ưu 1 đơn vị, đặc biệt khi i nhỏ, tốc độ chương trình bị ảnh hưởng rõ rệt. Vì vậy có thể dùng cùng kiểu tìm kiếm để tính $Dist[i]$; đáp án của bài chính là $Dist[1]$. Cách này tăng hiệu quả của điều kiện cắt tỉa 1, nhưng đồng thời việc tìm kiếm cho mỗi i cũng tốn thời gian.

5.9 So sánh kết quả chương trình

	1	2	3	4	5	6	7	8	9	10
Tìm kiếm đơn giản	0.00	0.00	0.50	T.O.	0.65	T.O.	T.O.	T.O.	T.O.	T.O.
Tìm kiếm tối ưu	0.01	0.01	0.10	T.O.	0.50	0.80	T.O.	0.55	0.30	0.80
Tối ưu thêm	0.01	0.01	0.02	0.03	0.00	0.02	0.02	0.01	0.02	0.02

Với phương pháp tìm kiếm này, nói chung có hai tối ưu khả thi rất hiệu quả:

1. Kiểm tra sớm xem có thể thành công hay không, tránh tìm kiếm thừa.
2. Mỗi lần kiểm tra cố gắng tận dụng nhiều nhất kết quả kiểm tra trước đó.

6 Tổng kết

Các ví dụ trong bài đều có thể áp dụng phương pháp tìm kiếm bộ phận để giải hiệu quả, và chúng có những điểm chung rõ rệt về mặt tư tưởng. Quá trình suy nghĩ tổng quát có thể mô tả bằng lời như sau:

Khó nghĩ ra thuật toán đa thức; tìm kiếm thông thường đơn giản không giải quyết được bài toán

↓

Muốn giảm lượng tìm kiếm

↓

Chọn các biến cần tìm kiếm

↓

Xét các biến còn lại có thể giải bằng thuật toán hiệu quả hay không

Nếu không, quay lại chọn biến tìm kiếm

↓

Xét liệu thuật toán có nâng cao hiệu suất hay không

Nếu không, quay lại chọn biến tìm kiếm

↓

Thông qua tối ưu thu được một phương pháp tìm kiếm hiệu quả

Các tối ưu thông thường gồm:

1. Chọn tốt các biến cần liệt kê, làm cho số biến liệt kê ít nhất có thể.
2. Tận dụng đầy đủ nhất kết quả trước đó, kiểm tra sớm và tránh tìm kiếm thừa.

Tìm kiếm bộ phận cũng có thể kết hợp với giải hệ phương trình, tham lam, quy hoạch động và các thuật toán hiệu quả khác.

Tìm kiếm bộ phận + thuật toán hiệu quả thể hiện sự kết hợp hữu cơ giữa tìm kiếm và các phương pháp khác, phát huy đầy đủ ưu điểm của cả hai và bù đắp nhược điểm của nhau. Đây là nguyên nhân chính tạo nên hiệu quả của nó. Vì vậy, trong các bài toán tìm kiếm, việc linh hoạt áp dụng tìm kiếm bộ phận thường có thể tạo ra hiệu quả bất ngờ.

Cần chú ý rằng dùng tìm kiếm bộ phận để giải bài toán tìm kiếm là một phương pháp tìm kiếm không thông thường. Tuy trong các ví dụ của bài, tìm kiếm bộ phận có nhiều ưu điểm vượt trội, nhưng không

thể cho rằng phương pháp thông thường nhất định kém hơn phương pháp không thông thường. Đa số bài toán tìm kiếm vẫn phù hợp với các phương pháp tìm kiếm thông thường. Chỉ khi nắm vững đặc điểm của tìm kiếm bộ phận và kết hợp nó nhuần nhuyễn với tìm kiếm thông thường, ta mới thật sự thu được một thuật toán tìm kiếm hiệu quả.

Tài liệu tham khảo

- ACM/ICPC Asia Regional Shanghai 1996, *Milk Bottle Data*.
- ZJU Monthly Contest, *Triangle Construction*.
- NOI 2003, *Phá trận liên hoàn* (智破连环阵).

Lời cảm ơn

Cảm ơn Zhejiang University Online Judge đã cung cấp mô tả bài *Milk Bottle Data* và *Triangle Construction*.

A Chương trình tham khảo cho Phá trận liên hoàn

Phần phụ lục gốc đưa ra một chương trình C/C++ tham khảo. Các chú thích trong mã dưới đây được dịch sang tiếng Việt; cấu trúc chương trình và tên biến được giữ theo bản trích xuất.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

const int maxm=100+2;
const int maxn=100+2;

int n,m,dist[maxm],MaxT[maxm][maxn];
bool reachable[maxm][maxn],*can[maxm][maxm];
/*
m = so vu khi cua nuoc B.
n = so bom cua nuoc A.
dist[i] = neu bom A duoc phep dung lap lai, so bom it nhat de tieu diet
cac vu khi B tu i den m.
MaxT[s][i] = bom i, neu bat dau no tu s, co the tieu diet den chi so lon nhat;
neu bom i khong tieu diet duoc s thi MaxT[s][i]=s-1.
reachable[i][j] = bom j cua nuoc A co the tieu diet vu khi i cua nuoc B hay khong.
can[s][t][i] = bom i cua nuoc A co the tieu diet vu khi B tu s den t hay khong.
*/
int answer,bestv[maxn];
/*
answer = so bom A it nhat can dung.
bestv = ghi lai day bom A duoc dung trong nghiem toi uu.
*/
```

```

int a[maxm],b[maxn];
bool vis[maxn],*g[maxm];
/*
a[i], b[i] dung cho ghep cap; lan luot ghi dinh o phia con lai duoc ghep voi
dinh i ben trai/phai, neu chua ghep thi bang 0.
vis[i] dung de danh dau trong ghep cap va tim kiem theo chieu rong.
g[i][j] = co canh tu dinh i ben trai den dinh j ben phai hay khong.
*/
void init()
{
    // Doc du lieu va tinh reachable.
    int x1[maxm],y1[maxm],x2[maxn],y2[maxn],i,j,R;
    scanf("%d%d%d",&m,&n,&R);
    for (i=1;i<=m;i++)
        scanf("%d%d",&x1[i],&y1[i]);
    for (i=1;i<=n;i++)
        scanf("%d%d",&x2[i],&y2[i]);
    for (i=1;i<=m;i++)
        for (j=1;j<=n;j++)
            reachable[i][j]=
                ((x1[i]-x2[j])*(x1[i]-x2[j])+
                 (y1[i]-y2[j])*(y1[i]-y2[j])<=R*R);
}

void preprocess()
{
    // Khoi tao, tinh can va MaxT.
    int s,t,i;
    for (i=1;i<=m;i++)
        g[i]=new bool[maxn];
    for (s=1;s<=m;s++)
        for (t=s;t<=m;t++)
            can[s][t]=new bool[maxn];
    for (s=1;s<=m;s++)
    {
        for (i=1;i<=n;i++)
            can[s][s][i]=reachable[s][i];
        for (t=s+1;t<=m;t++)
            for (i=1;i<=n;i++)
                can[s][t][i]=can[s][t-1][i] && reachable[t][i];
        for (i=1;i<=n;i++)
        {
            MaxT[s][i]=s-1;
            for (t=s;t<=m;t++)
                if (can[s][t][i])
                    MaxT[s][i]=t;
        }
    }
}

```

```

// Tinh dist.
dist[m+1]=0;
for (s=m;s>=1;s--)
{
    t=s-1;
    for (i=1;i<=n;i++)
        if (MaxT[s][i]>t)
            t=MaxT[s][i];
    dist[s]=1+dist[t+1];
}
}

bool find(int v)
{
    // Thuat toan Hungary tim duong tang.
    for (int i=1;i<=n;i++)
        if (g[v][i] && !vis[i])
        {
            vis[i]=true;
            if (b[i]==0 || find(b[i]))
            {
                a[v]=i;
                b[i]=v;
                return true;
            }
        }
    return false;
}

void search(int used,int s)
{
    // Trang thai: da dung used bom A; cac vu khi truoc s da bi pha huy.

    if (used+dist[s]>=answer) // Toi uu 1: cat tia toi uu.
        return;
    if (s==m+1)
    {
        // Tat ca vu khi B da bi pha huy: cap nhat nghiem toi uu roi quay lui.
        answer=used;
        memcpy(bestv,a,sizeof(a));
        return;
    }
    int t,maxL,tempa[maxm],tempb[maxn],op,cl,queue[maxn],k,i;
    // Tim kiem theo chieu rong de tinh do dai lon nhat maxL cua doan tiep theo.
    memset(vis,false,sizeof(vis));
    maxL=s-1;
    op=cl=0;
    for (i=1;i<=n;i++)

```

```

        if (b[i]==0)
        {
            vis[i]=true;
            queue[++op]=i;
        }
    while (cl<op)
    {
        k=queue[++cl];
        if (MaxT[s][k]>maxL)
            maxL=MaxT[s][k];
        for (i=1;i<=used;i++)
            if (g[i][k] && !vis[a[i]])
            {
                vis[a[i]]=true;
                queue[++op]=a[i];
            }
    }
    if (maxL==s-1)
        return;

    used++;
    memcpy(tempa,a,sizeof(a));
    memcpy(tempb,b,sizeof(b));
    memset(vis,false,sizeof(vis));
    g[used]=can[s][maxL];
    // Mo rong duong xen ke.
    find(used);
    // Liet ke do dai doan tiep theo tu lon den nho.
    for (t=maxL;t>=s;t--)
    {
        g[used]=can[s][t];
        search(used,t+1);
    }
    memcpy(a,tempa,sizeof(a));
    memcpy(b,tempb,sizeof(b));
}

void out()
{
    // In ket qua.
    printf("%d\n",answer);
    for (int i=1;i<=answer;i++)
        printf("%d ",bestv[i]);
    printf("\n");
}

int main()
{

```

```
freopen("zplhz.in","r",stdin);
freopen("zplhz.out","w",stdout);
init();
preprocess();
answer=1000000000;
memset(a,0,sizeof(a));
memset(b,0,sizeof(b));
search(0,1);
out();
return 0;
}
```